Program:

A program is a prepared sequence of instructions to accomplish a defined task.

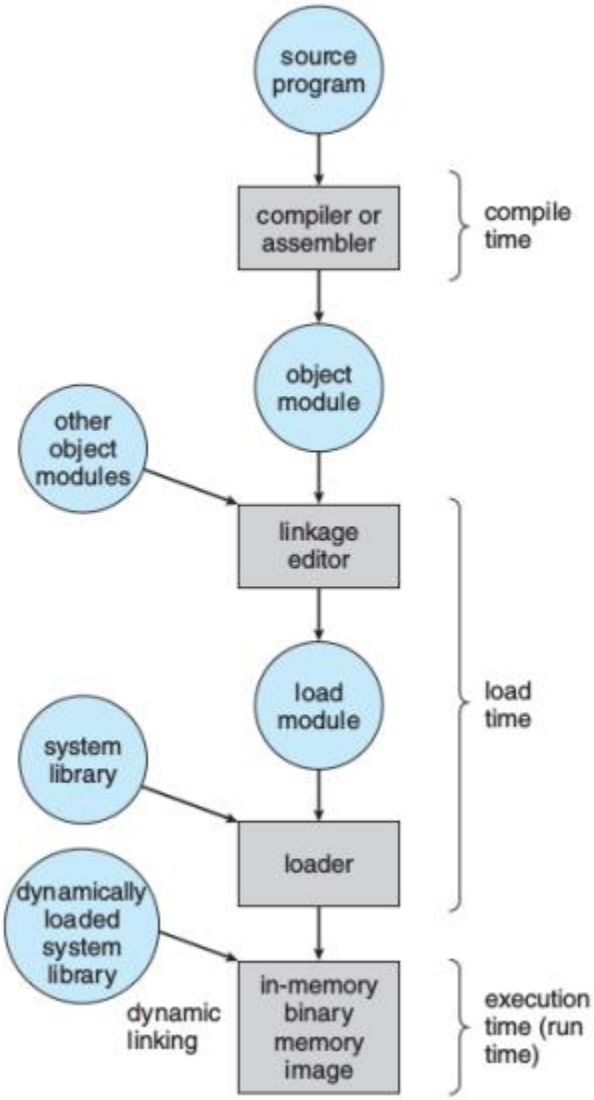
The C compiler

1. Translates each source file into an object file.

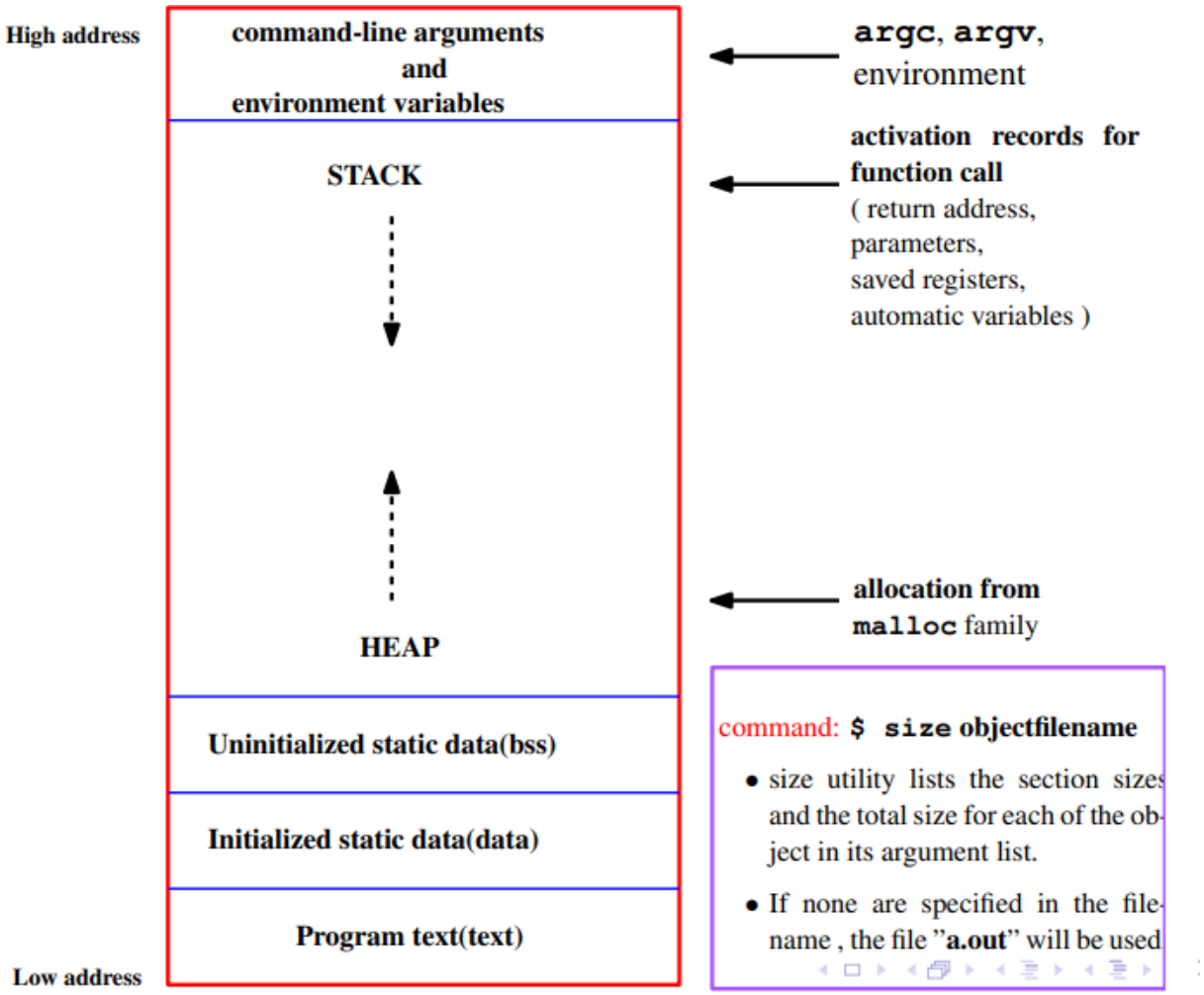
2. The compiler then links the individual object files with the necessary libraries to produce an executable module.

3. When a program is run or executed, the operating system copies the executable module into a program image in main memory

Multistage Processing of a User Program



Layout of a Program Image in Main Memory



Process

process is an instance of a program that is executing (A program which is dispatched from ready state and are scheduled for execution)

When does program become process?

☞The operating system reads the program into memory.

☞ The allocation of memory for the program image is not enough to make the program a process.

☞ The process must have an ID (the process ID) so that the operating system can distinguish among individual processes. The process state indicates the execution status of an individual process.

☞ The operating system keeps track of the process IDs and corresponding process states and uses the information to allocate and manage resources for the system.

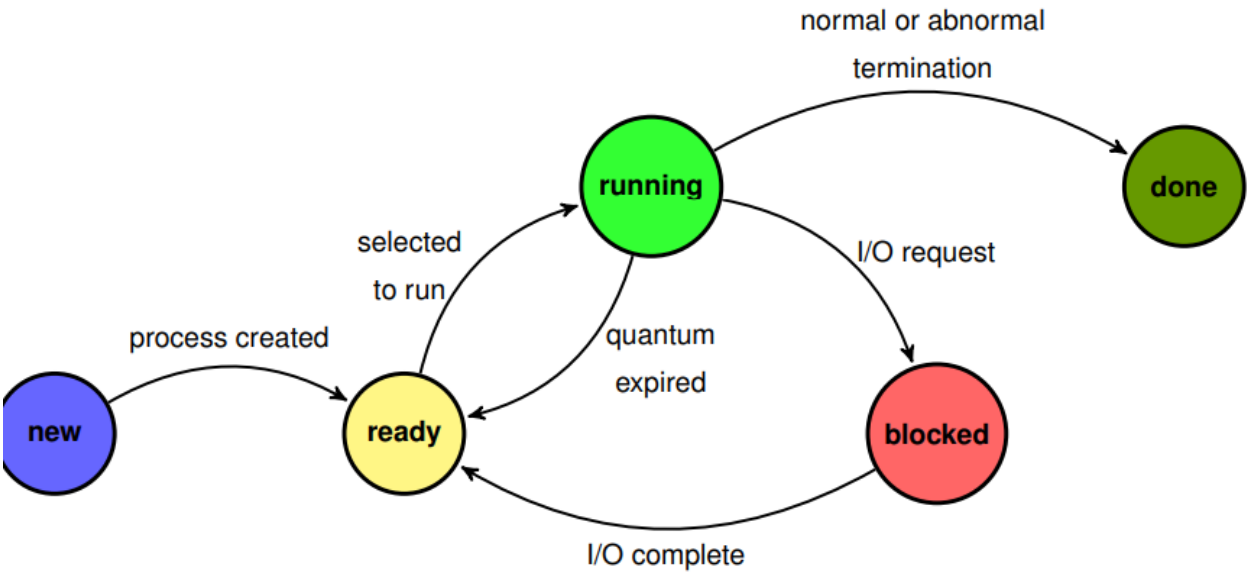
☞ The operating system also manages the memory occupied by the processes and the memory available for allocation.

☞ When the operating system has added the appropriate information in the kernel data structures and has allocated the necessary resources to run the program code, the program has become a process.

☞ A process has an address space (memory it can access) and at least one flow of control called a thread.

☞ The variables of a process can either remain in existence for the life of the process (static storage) or be automatically allocated when execution enters a block and deallocated when execution leaves the block (automatic storage)

Process State



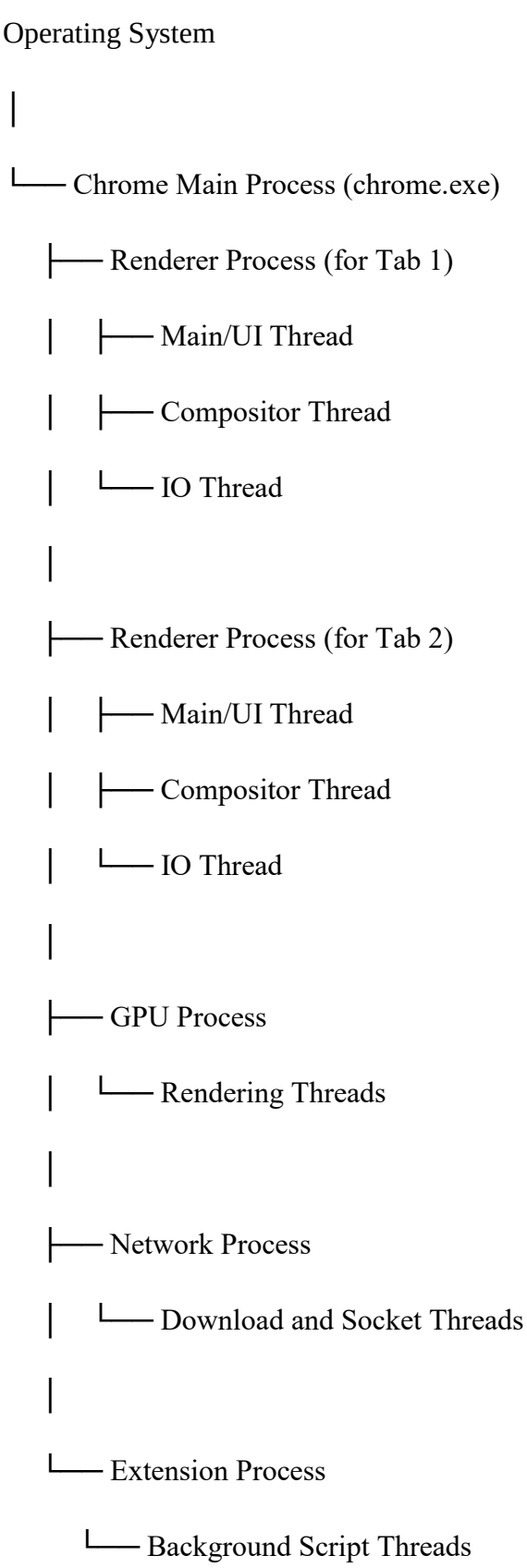
Thread :

A thread is a single sequence stream within a process. Threads are also

called **lightweight processes** as they possess some of the properties of processes.

A **thread** is a **lightweight subpart** of that process that performs a specific task.

Example:


Operating System
Chrome Main Process (chrome.exe)
Renderer Process (for Tab 1)
Main/UI Thread
Compositor Thread
IO Thread
Renderer Process (for Tab 2)
Main/UI Thread
Compositor Thread
IO Thread
GPU Process
Rendering Threads
Network Process
Download and Socket Threads
Extension Process
Background Script Threads

Tracking ongoing processes

1. ps (Process status) can be used to see/list all the running processes.

For single-process information, ps along with process id is used

-a: Shows information about all users

-x: Shows information about processes without terminals

-u: Shows additional information

-f: Displays full information For a running program

ps -al : Shows information about all process

ps -ef: Full-format listing

ps aux: BSD-style detailed view

Outputs

Pidof finds the process id's (pid Fields described by ps are described as:

- UID: User ID that this process belongs to (the person running it)
- PID: Process ID
- PPID: Parent process ID (the ID of the process that started it)
- C: CPU utilization of process
- STIME: Process start time
- TTY: Terminal type associated with the process
- TIME: CPU time is taken by the process
- CMD: The command that started this process
- VSZ: Total amount of virtual memory used by the process
- RSS: Actual physical memory (RAM) currently occupied by the process (in kilobytes). This excludes memory swapped out to disk.

2. Kill: Used to stop a process

Syntax:: $ kill process-id Example:: $ kill 4578

Process Identification

☞UNIX identifies processes by a unique integral value called the process ID.

☞ Each process also has a parent process ID, which is initially the process ID of the process that created it.

☞ If this parent process terminates, the process is adopted by a system process so that the parent process ID always identifies a valid process.

getpid and getppid Function

The getpid function returns the process ID .

☞ The getppid function returns the parent process ID.

☞ The pid_t is an unsigned integer type that represents a process ID.

```
#include <unistd.h>
pid_t getpid(void);
pid_t getppid(void);
```

Neither the getpid nor the getppid functions can return an

error.

Example

```
#include<stdio.h>
#include<unistd.h>
int main (void)
{
printf("I am process %ld\n", (long)getpid());
printf("My parent is %ld\n", (long)getppid());
return 0;
 }
```

UNIX Process Creation and fork

- ✍ A process can create a new process by calling fork.
- ✍ The calling process becomes the parent, and the created process is called the child.
- ✍ The fork function copies the parent's memory image so that the new process receives a copy of the address space of the parent.
- ✍ Both processes continue at the instruction after the fork statement (executing in their respective memory images).
- ✍ fork() prototype

```
#include <unistd.h>
pid_t fork(void);
```

✍ fork() function returns

(1) returns 0 to the child

(2) returns the child’s process ID to the parent.

(3) When fork fails, it returns -1

Note:: The fork function return value is the critical characteristic that allows the parent and the child to distinguish themselves and to execute different code.

Example:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid > 0)
        printf("Parent Process: PID = %d, Child PID = %d\n", getpid(), pid);
    else if (pid == 0)
        printf("Child Process: PID = %d, Parent PID = %d\n", getpid(),
getppid());
    else
        printf("Fork failed!\n");
    return 0;
}
```

Example 1:

```
#include <stdio.h>
#include <unistd.h>
int main() {
    printf("Before fork\n");
    fork();
    printf("After fork\n");
```

```
    return 0;
}
```

o/p: Before fork

After fork

After fork

Explanation

P #before fork

(fork)

C P #After fork

Or

Parent process

```
#include <stdio.h>
#include <unistd.h>
int main() {
    printf("Before fork\n");
    fork();
    printf("After fork\n");
    return 0;
}
```

Child process

```
#include <stdio.h>
#include <unistd.h>
int main() {
    printf("After fork\n");
    return 0;
}
```

Example2:

```
#include <stdio.h>
#include <unistd.h>
int main() {
    fork();    // 1st fork
    fork();    // 2nd fork
    printf("Process: PID = %d, PPID = %d\n", getpid(), getppid());
    return 0;
```

}

Explanation:

Parent

```
#include <stdio.h>
#include <unistd.h>
int main() {
    fork();    // 1st fork
    fork();    // 2nd fork
    printf("Process: PID
= %d, PPID = %d\n",
getpid(), getppid());
    return 0;
}
```

Child 0

```
#include <stdio.h>
#include <unistd.h>
int main() {
    fork();    // 2nd fork
    printf("Process: PID
= %d, PPID = %d\n",
getpid(), getppid());
    return 0;
}
```

Child 2

```
#include <stdio.h>
#include <unistd.h>
int main() {
    printf("Process: PID
= %d, PPID = %d\n",
getpid(), getppid());
    return 0;
}
```

Child 1

```
#include <stdio.h>
#include <unistd.h>
int main() {
    printf("Process: PID
= %d, PPID = %d\n",
getpid(), getppid());
    return 0;
}
```

Process Tree Visualization

Parent (P0)

├── Child1 (P1)

│ └── Grandchild (P3)

└── Child2 (P2)

or

P

C1 P

C2 C1 C3 P

Remember: Total = 2^n processes (for *n* successful forks).

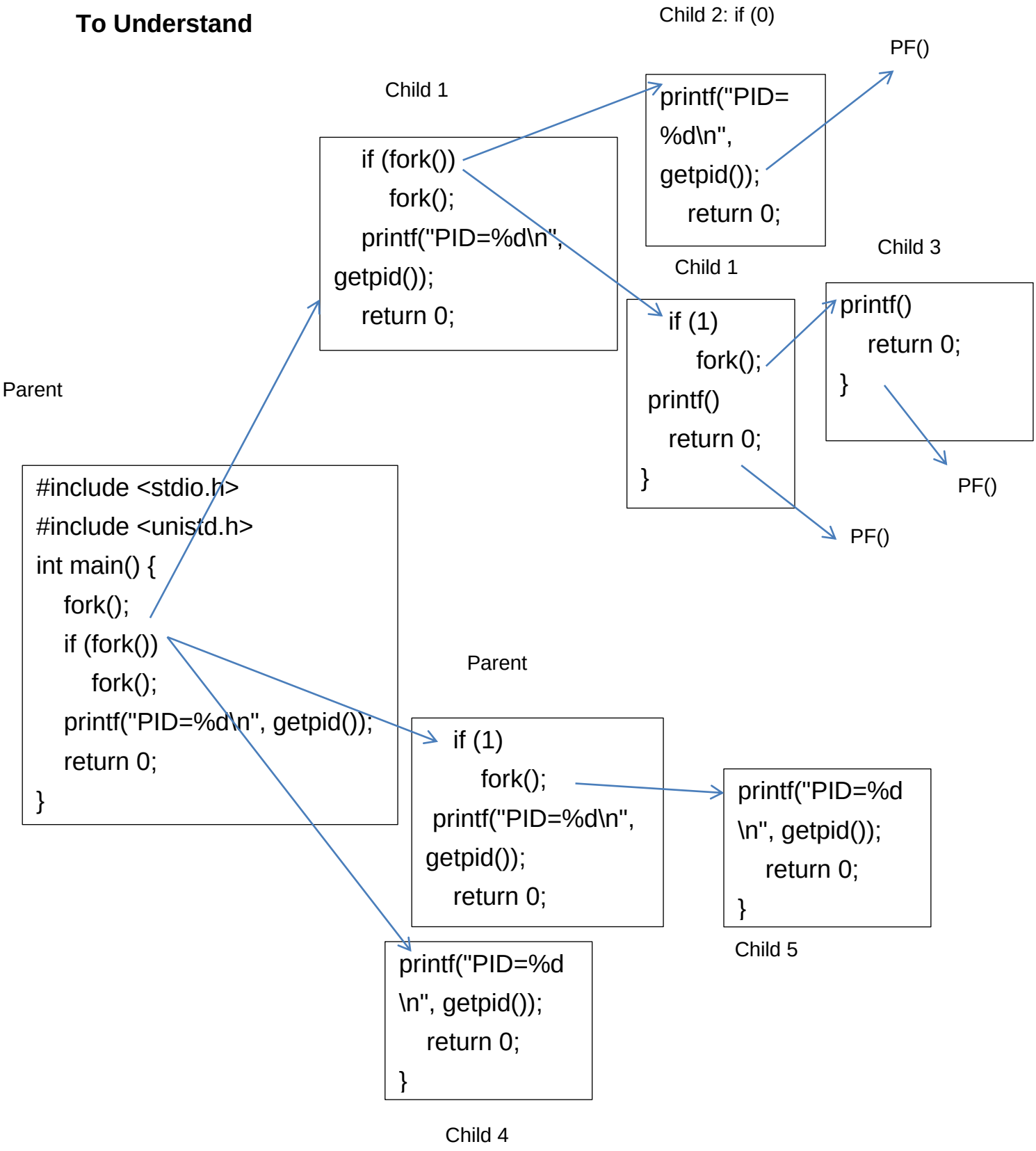
Example:

```
#include <stdio.h>
#include <unistd.h>
int main() {
    fork();
    if (fork())
        fork();
    printf("PID=%d\n", getpid());
    return 0;
}
```

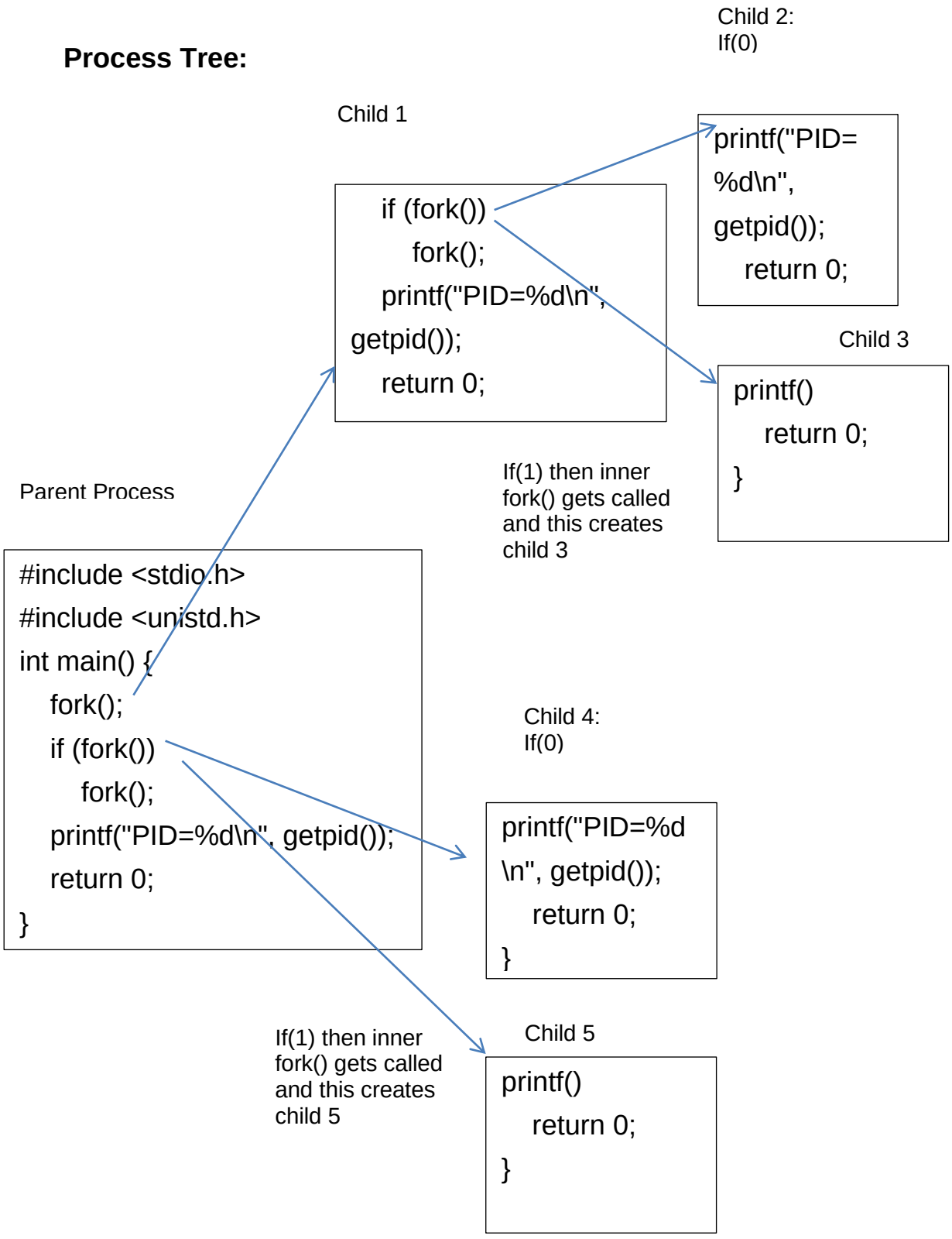
o/p: PID=313
PID=314
PID=311
PID=312
PID=315
PID=316

Total 6 processes

To Understand



Process Tree:



- For `A && B`:
  - If `A` is **false (0)**, then `B` is **not evaluated**, because the whole expression can never be true.

- For `A || B`:
  - If `A` is **true (non-zero)**, then `B` is **not evaluated**, because the whole expression is already true.

1||x = 1

0||x=x

Example

```
#include <stdio.h>
#include <unistd.h>
int main() {
    fork();
    if (fork()||fork())
        fork();
    printf("PID=%d\n", getpid());
    return 0;
}
```

o/p:

PID=322

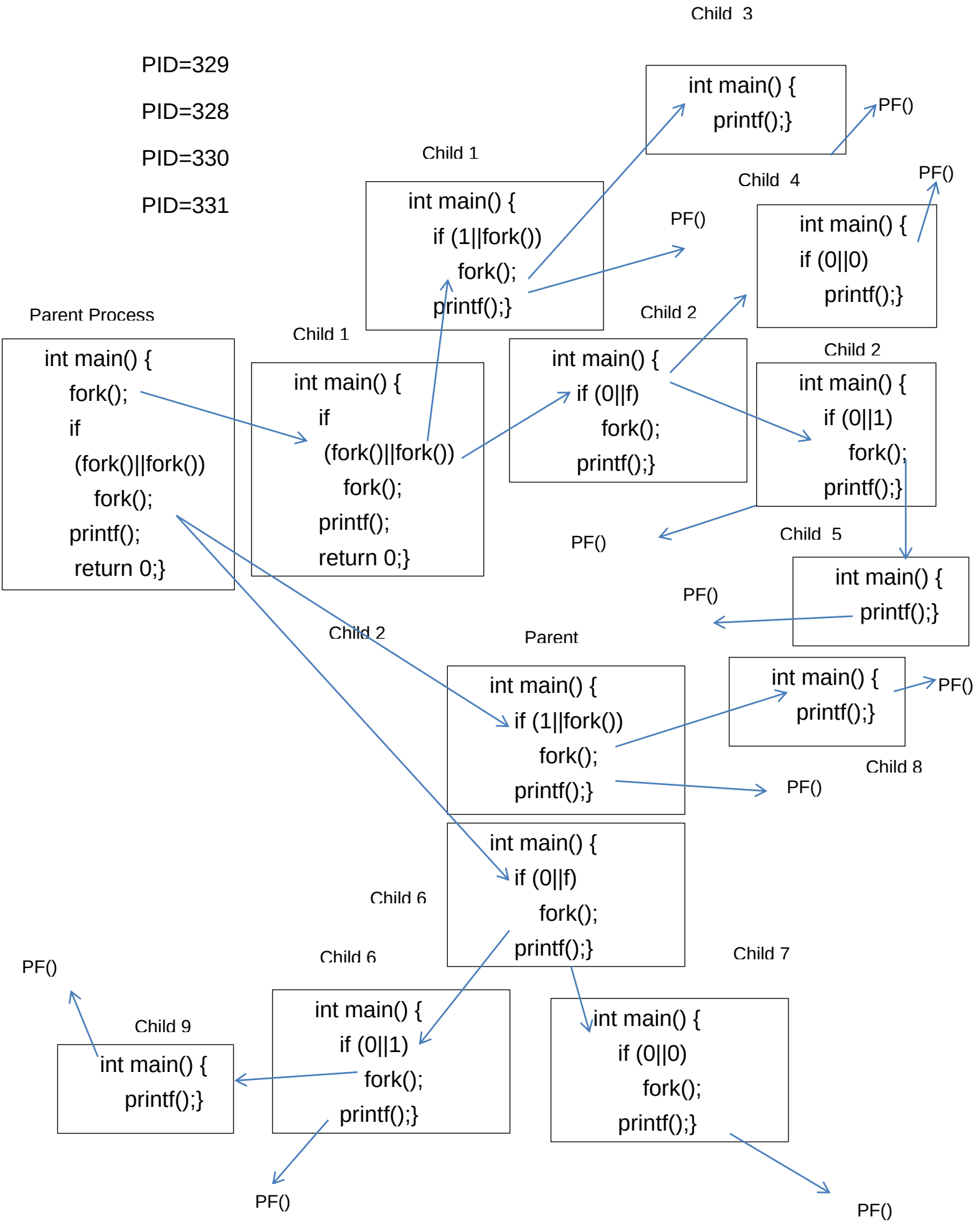
PID=323

PID=324

PID=325

PID=327

PID=326


PID=329
PID=328
PID=330
PID=331
Parent Process
int main() {
fork();
if
(fork()||fork())
fork();
printf();
return 0;}
Child 1
int main() {
if
(fork()||fork())
fork();
printf();
return 0;}
Child 1
int main() {
if (1||fork())
fork();
printf();}
Child 2
int main() {
if (0||f)
fork();
printf();}
Child 3
int main() {
printf();}
Child 4
int main() {
if (0||0)
printf();}
Child 2
int main() {
if (0||1)
fork();
printf();}
Child 5
int main() {
printf();}
Child 2
Parent
int main() {
if (1||fork())
fork();
printf();}
Child 6
int main() {
if (0||f)
fork();
printf();}
Child 6
int main() {
if (0||1)
fork();
printf();}
Child 7
int main() {
if (0||0)
fork();
printf();}
Child 8
int main() {
printf();}
Child 9
int main() {
printf();}
PF()
PF()
PF()
PF()
PF()
PF()
PF()
PF()
PF()

0&&x = 0

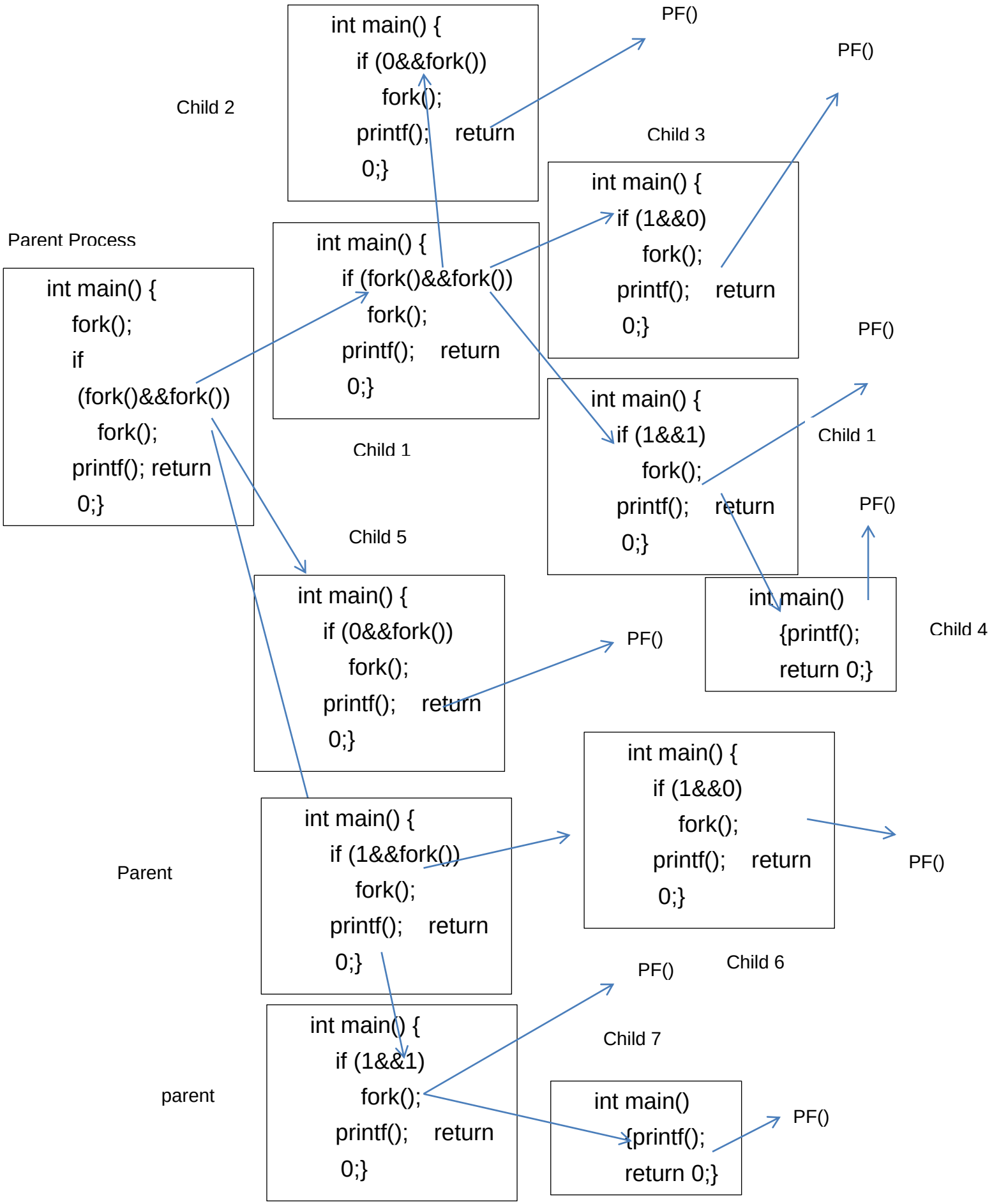
1&&x =x

Example

```
#include <stdio.h>
#include <unistd.h>
int main() {
    fork();
    if (fork() && fork())
        fork();
    printf("PID=%d\n", getpid());
    return 0;
}
```

o/p:

```
PID=350
PID=351
PID=348
PID=349
PID=352
PID=353
PID=355
PID=354
```


Parent Process
int main() {
fork();
if
(fork()&&fork())
fork();
printf(); return
0;}
Child 1
int main() {
if (fork()&&fork())
fork();
printf(); return
0;}
Child 2
int main() {
if (0&&fork())
fork();
printf(); return
0;}
PF()
Child 3
int main() {
if (1&&0)
fork();
printf(); return
0;}
PF()
Child 1
int main() {
if (1&&1)
fork();
printf(); return
0;}
PF()
Child 4
int main()
{printf();
return 0;}
Child 5
int main() {
if (0&&fork())
fork();
printf(); return
0;}
PF()
Parent
int main() {
if (1&&fork())
fork();
printf(); return
0;}
Child 6
int main() {
if (1&&0)
fork();
printf(); return
0;}
PF()
parent
int main() {
if (1&&1)
fork();
printf(); return
0;}
Child 7
int main()
{printf();
return 0;}
PF()

Example:

```
#include <stdio.h>
#include <unistd.h>
int main() {
    for (int i = 0; i < 3; i++)
        fork();
    printf("Hi");
    return 0;
}
```